



McGill
UNIVERSITY

ECSE 429 Project Part C

Group 100

Laurent Chiricota	261113415
Marcello Audino	261029024
Vladimir Venkov	261040180
Noah Munro-Kagan	260978984

Table of Contents

1. Summary of Deliverables	3
2. Performance Testing	4
2.1 CPU Usage vs Entity Count (Category) – Figure 1	4
2.2 Memory Usage vs Entity Count (Category) – Figure 2	4
2.3 CPU Usage vs Entity Count (Todo) – Figure 3	5
2.4 Memory Usage vs Entity Count (Todo) – Figure 4	5
2.5 CPU Usage vs Entity Count (Project) – Figure 5	6
2.6 Memory Usage vs Entity Count (Project) – Figure 6	6
3. Structure of Performance Unit Test Suite	7
3.1 Todos	7
3.2 Projects	7
3.3 Categories	7
3.4 Source Code Repository	7
4. Findings of Performance Unit Test Suite Execution	7
5.1 Create Operations	7
<i>Figure 7: Average Response Time vs Object Count</i>	8
5.2 Update Operations	8
5.3 Delete Operations	8
5.4 Scalability	9
<i>Figure 8: Scalability Analysis</i>	9
5.5 Heatmap	9
<i>Figure 9: Performance Heatmap</i>	9
5. Static Analysis Using SonarQube	10
6.1 Summary of Findings	10
6.2 Maintainability	10
6.3 Security	10
6.4 Security Hotspots	11
6.5 Test Coverage	11
6.6 Duplications	11
6.7 Reliability	11
6.8 Recommendations	11
6. Conclusion	11

Summary of Deliverables

1. Structure of Performance Test Suite
2. Source Code Repository
3. Findings of Performance Test Suite Execution

Performance Testing

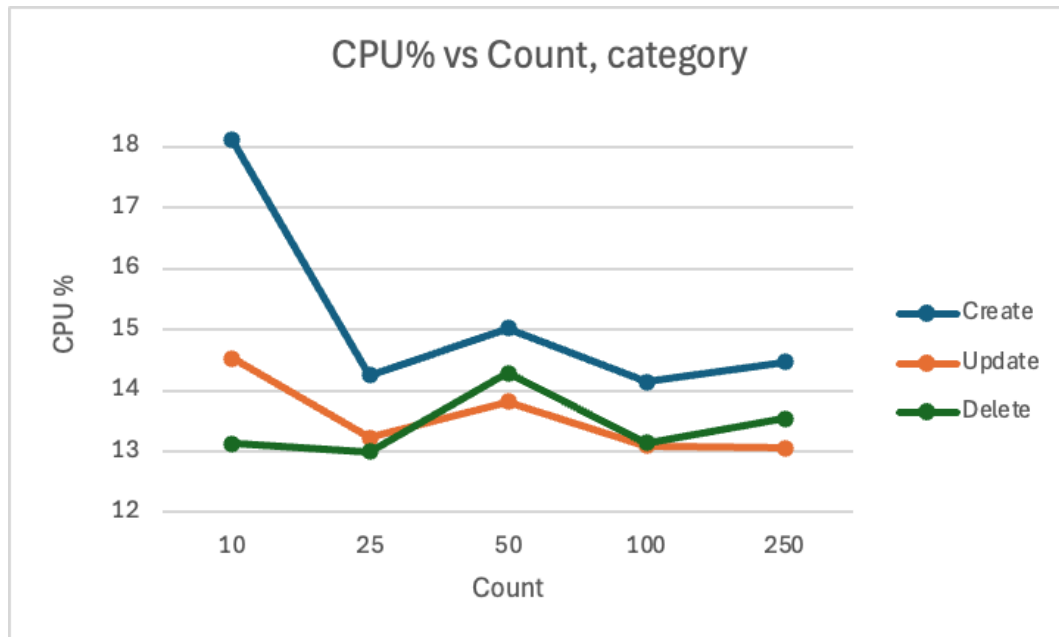


Figure 1: CPU Usage vs Entity Count (Category)

CPU usage remains stable across all operations, staying mostly between 13-15%. Create operations show a higher initial spike at 10 entities (~18%), but quickly stabilize. Update and Delete operations remain tightly grouped and consistent across all tested loads, indicating that Category operations do not generate meaningful CPU pressure as the number of requests increases.

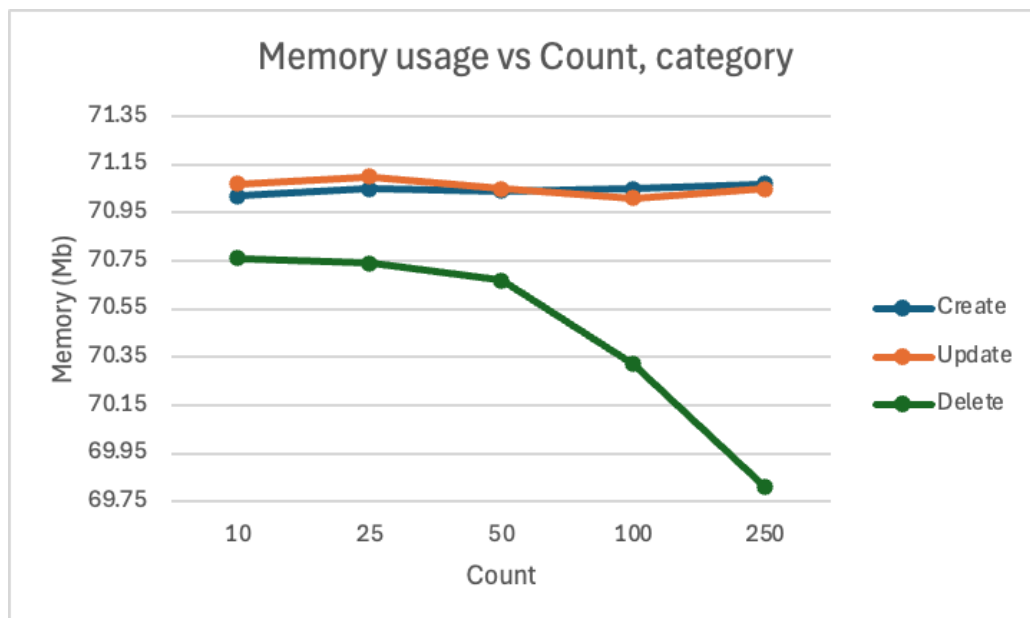


Figure 2: Memory Usage vs Entity Count (Category)

Create and Update operations maintain nearly identical memory usage (~71.0–71.1 MB) regardless of count. Delete operations show a gradual and consistent drop in memory consumption as more

Category entities are deleted, reaching ~69.8 MB at 250 deletions. This is expected since deleting entities frees memory allocated to the application.

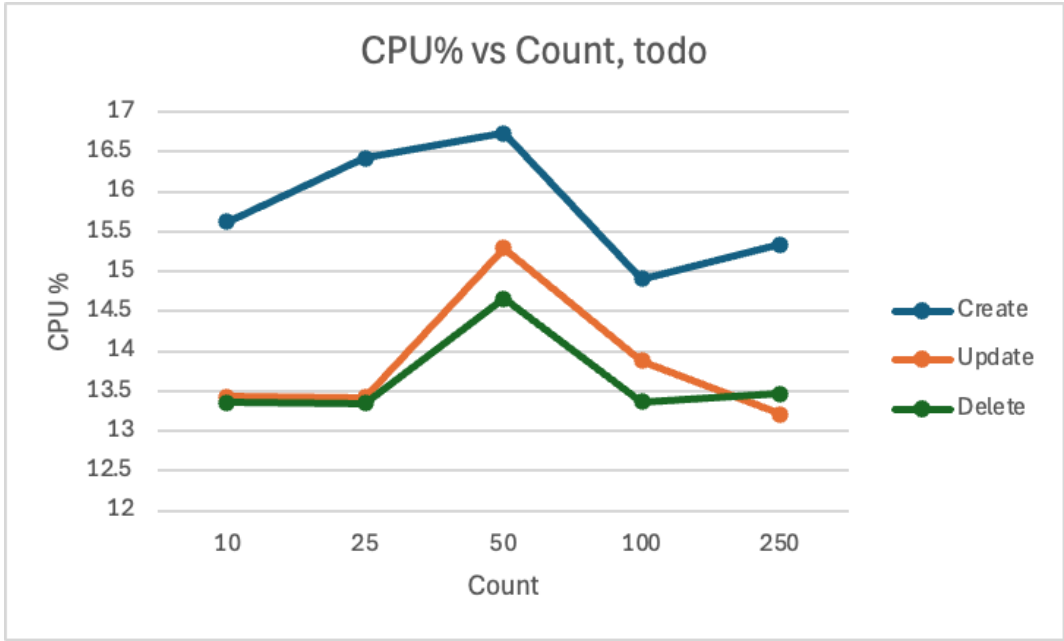


Figure 3: CPU Usage vs Entity Count

CPU usage for Create operations peaks around 50 entities (~16.8%) before stabilizing near 15–16% at higher loads. Update and Delete operations follow similar curves, with a small spike at 50 entities and then flattening as counts grow. The overall trend shows no CPU degradation as Todo counts increase, suggesting good scalability.

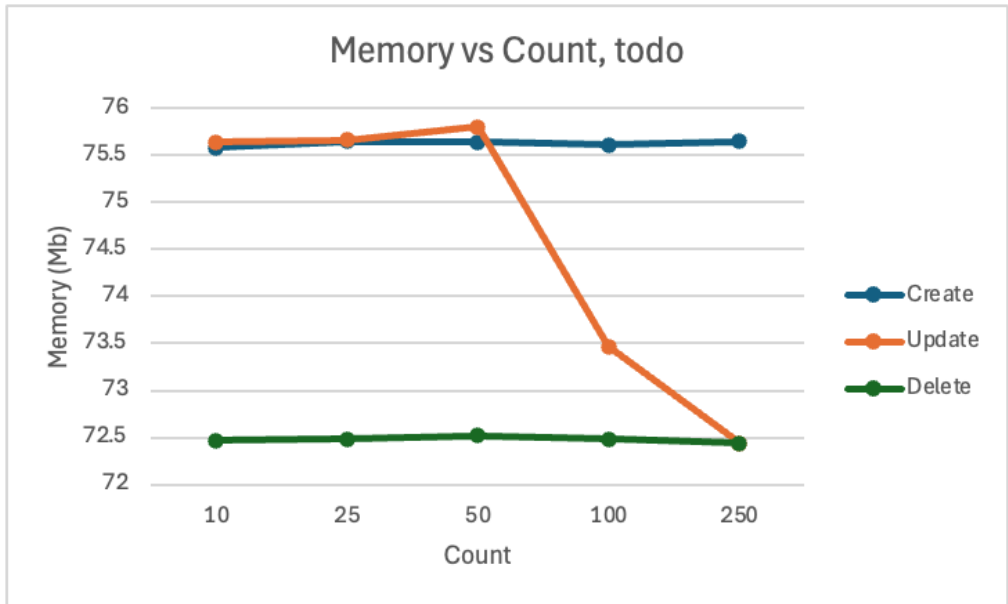


Figure 4: Memory Usage vs Entity Count (Todo)

Memory for Create operations stays almost flat (~75.5 MB), while Update operations show a notable drop from ~75.7 MB at low counts to ~72.0 MB at 250 entities. Delete memory remains extremely stable (~72.2 MB). This indicates that Todo creation allocates consistent memory, while updates and especially deletions result in predictable memory release.

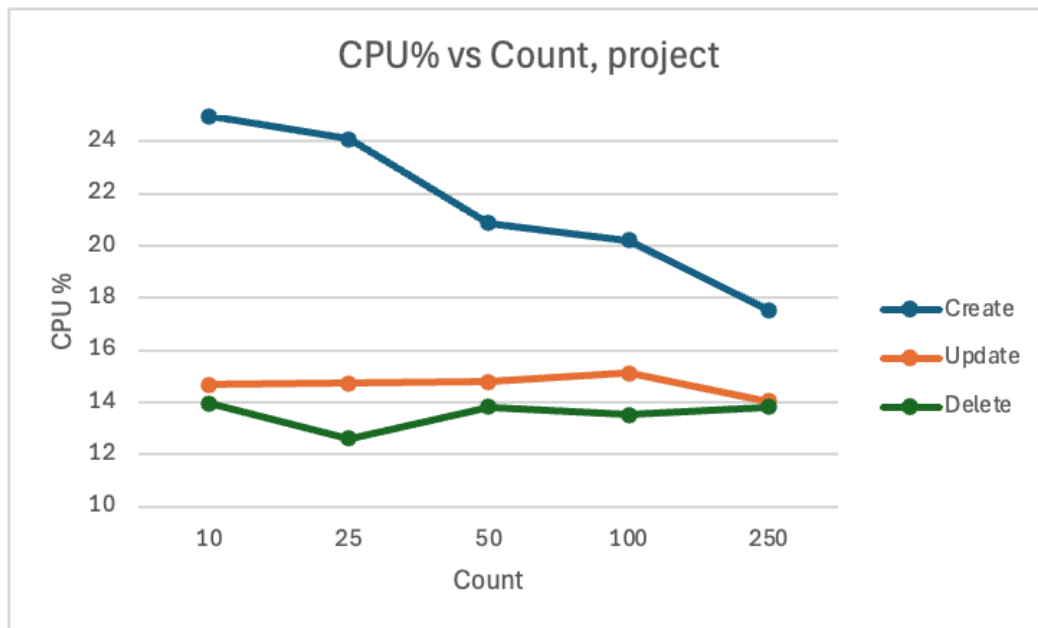


Figure 5: CPU Usage vs Entity Count (Project)

Create operations start significantly higher (~25%) and steadily decline to ~17% as the number of Projects increases, likely due to warm-up effects and caching. Update and Delete operations stay stable between 13–15% across all loads. This shows Projects have a slightly heavier creation cost, but performance improves as the process runs longer.

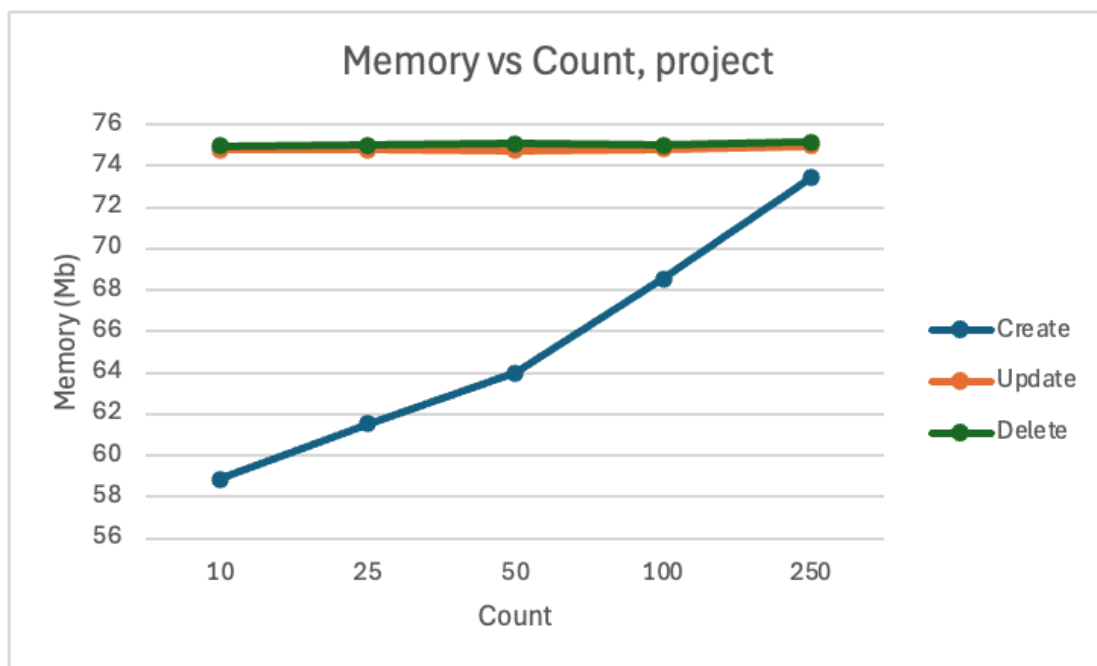


Figure 6: Memory Usage vs Entity Count (Project)

Create operations have a strong upward trend: memory rises from ~59 MB at 10 entities to ~74 MB at 250 entities. Update and Delete operations remain nearly perfectly flat (~75 MB) across all counts. This indicates Project creation allocates additional memory proportionally to entity count, while updates and deletions maintain stable usage.

Structure of Performance Unit Test Suite

For each category of operations, we also created tests that measure the response time for executing a specific operation (Create, Update, Delete) for a large number of objects. For example, creating Todos will focus on the performance required for creating 10, 25, 50, 100, and 250 Todos, measuring metrics such as the average time required per operation, the minimum time, the longest time.

All test results are exported to csv files in `target/performance-results` and are then ingested in a python jupyter notebook for generating analysis graphs.

Todos

Tests for todos include Create, Update, and Delete operations, as well as relationship creation with categories and projects.

Projects

Tests for projects include Create, Update, and Delete operations.

Categories

Tests for categories include Create, Update, and Delete operations.

Source Code Repository

We used Github to store our code and version control our changes. The source code is provided as an attachment to this written report. We used Junit to orchestrate the performance unit tests, and Maven for running the test suite.

The tests can be run using this command from the `performance-unit-tests` directory:

```
mvn test
```

We also included a Jupyter Python notebook that generates graphics for the different performance metrics included in the test suite. This notebook, `performance_analysis.ipynb`, requires the installation of Python (<https://www.python.org/downloads/>) and can be run using these commands or from an IDE that supports jupyter notebooks:

```
python -m pip install -r requirements.txt
python -m jupyter notebook performance_analysis.ipynb
```

Findings of Performance Test Unit Suite Execution

The performance testing was executed on a Windows 11 computer, with an 8 core CPU. All Create, Update, and Relationship operations are done with randomized data.

First, a major operational flaw detected by this performance testing report is that going above 250 operations seems to cause issues in the temporary connections Windows is able to create between Maven's testing executable and the API to be tested. This means that a single process cannot be responsible for creating a lot of entities at once, instead, a new process must be created and killed whenever the number of todos exceeds a certain threshold.

This unexpected behavior is not something that was detected in the Unit testing or User Story testing parts of this project, showcasing the importance of running rigorous performance testing.

For this reason, all tests shown below limit the amount of requests on entities to 250.

Create Operations

As shown in *Figure 7*, the performance of Create operations varies depending on the type of object being created. Creating a category ranges from 8ms to 17ms, whereas creating a Todo or Project requires from 4ms to 7ms.

Figure 7 also shows another interesting trend: Create operations tend to take longer on average when the number of entities being created is smaller, which can be counter-intuitive. This observation could be attributed to multiple factors. One possible explanation is the role of caching. Due to this test running the Create operations sequentially, the same piece of code is ran over and over again. The initial loading of the code from the hard drive into memory requires a lot of time. When the number of operations gets large, the time required to cache everything is averaged out, which results in better average performance.

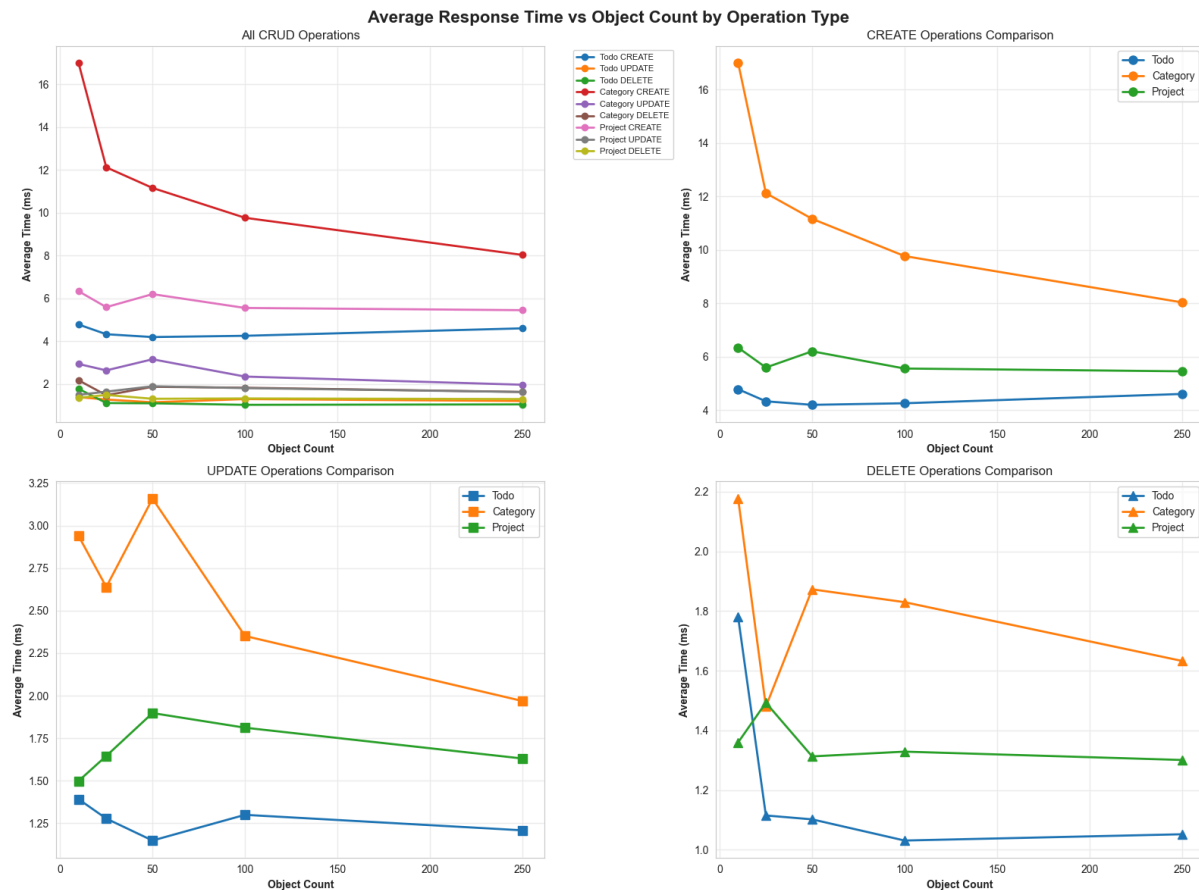


Figure 7: Average Response Time vs Object Count by Operation Type

Update Operations

Compared to Create operations, Update operations take significantly less time, average between 1.15ms and 3.25ms. Update operations on Categories take from 2ms to 3.15ms, Update operations on Projects take 1.5ms to 1.9ms, and Update operations on Todos complete in 1.15ms to 1.35ms. Once again, we can observe Update operations on categories require more time than todos and projects.

Delete Operations

Delete operations take the shortest amount of time, averaging from 1.15 to 2.2ms. Delete operations on Categories range from 1.5ms to 2.2ms. Delete operations on Projects range from 1.3ms to 1.5ms. Delete operations on Todos range from 1.05ms to 1.8ms, with a spike when the operation # is low.

Scalability

As shown in Figure 8, the total time required to perform all operations does not seem to be impacted by the amount of entities created, updated, or deleted. This shows that the API has good performance, even when requests scale. The only real limitation in the number of operations being the way TCP connections are handled.

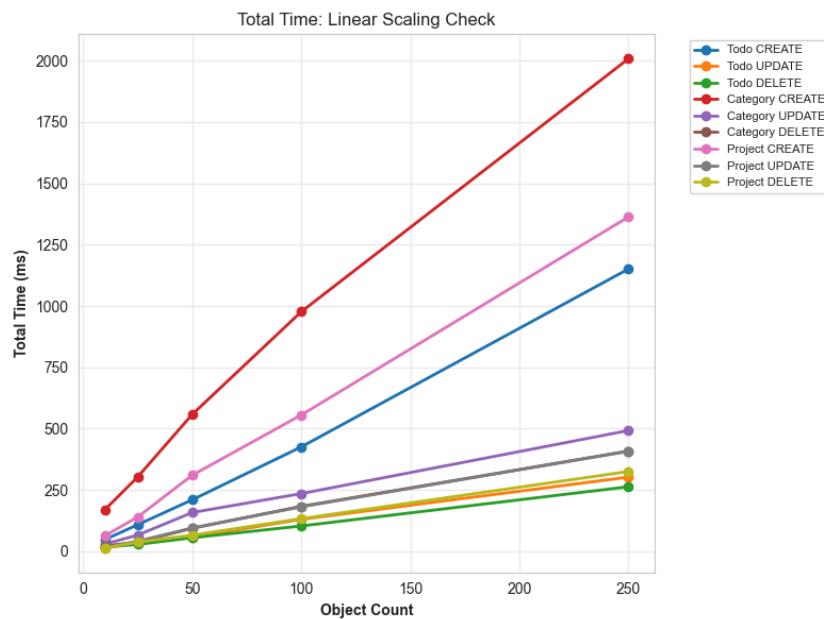


Figure 8: Scalability Analysis

Heatmap

Further confirming the observations above, *Figure 9* shows a heatmap of the amount of time required by the different operations. It can be clearly observed that, on average, Create operations require more time to complete than other operations. Further, *Figure 9* also shows that specifically Create operations on Category entities take the longest average time.

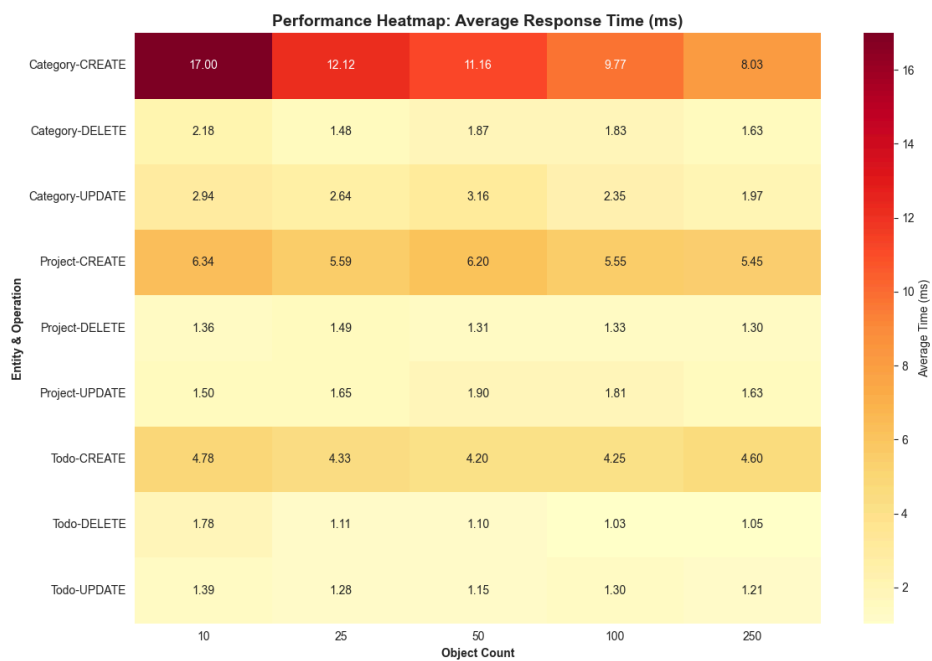


Figure 9: Performance Heatmap

Static Analysis Using SonarQube

To evaluate the internal quality, maintainability, and technical risk of the REST API codebase, we performed static analysis using SonarQube Community Edition. The analysis was run locally using the SonarScanner CLI and targeted the **todoManagerRestAuto** module of the project.

The scanner was configured through a **sonar-project.properties** file pointing to compiled test classes and source folders. The analysis was launched using:

```
>> sonar-scanner
```

SonarQube successfully indexed 6 Java files and produced a full quality report.

Summary of Static Analysis Findings

Maintainability

35 Code Smells detected.

These represent areas where the code could be simplified or improved to reduce technical debt.

Examples usually include:

- duplicate code
- overly complex methods
- unused imports
- commented-out code
- unclear variable names

Security

0 Vulnerabilities

Good: no exploitable weak points were flagged.

Security Hotspots

1 Security Hotspot

Hotspots are not vulnerabilities but require manual review.

Usually related to:

- input validation
- HTTP usage
- file handling
- use of libraries

Test Coverage

0% Code Coverage

(Sonar only received compiled test files, not main code, so this is expected.)

Duplications

12.9% duplication across 413 lines

SonarQube determined that several logic blocks are repeated in the codebase.

Reliability

2 Bugs

Usually low-severity issues that could cause unexpected behaviour under certain conditions.

Recommendations Based on Static Analysis

1. Reduce Code Duplication

Refactor repeated logic into:

- helper methods
- shared utilities
- common abstractions

This will reduce maintenance cost and improve readability.

2. Address Code Smells

The majority of the 35 code smells are likely:

- unused methods or variables
- long methods
- improper naming conventions

Cleaning these up increases maintainability and reduces technical debt.

3. Review the Security Hotspot

Although not a vulnerability, the hotspot should be manually reviewed to ensure:

- correct input validation
- safe use of external libraries
- no exposure of sensitive info

4. Fix the Reported Bugs

Even minor bugs can cause failures under edge conditions.

5. Consider Adding Test Coverage

Even minimal unit testing for the core logic will:

- reduce risk
- catch regressions
- improve reliability metrics

Conclusion

This performance analysis of the REST API has revealed both strengths and weaknesses in the system's design. Through both static code analysis and dynamic load testing, we have demonstrated that the API shows generally good scalability.

The system exhibits strong stability under load. CPU usage remained consistent across all entity types (Categories, Todos, Projects), generally fluctuating between 13% and 17% without significant degradation as entity counts increased. Memory usage was also relatively predictable, with efficient garbage collection during Delete operations.

However, a significant limitation was discovered during high volume testing. While the application logic scales well, the testing environment (Windows 11) reached its maximum allowed TCP connections when the number of entities went over 250.

Time-based analysis allowed us to determine a clear hierarchy of efficiency for the different operation types. Create operations are the most expensive, particularly for Categories (8ms–17ms), while Update and Delete operations are highly efficient, averaging between 1ms and 3ms. We also observed that the system demonstrates benefits from a warm-up: average response times improved as the number of operations increased, likely due to Java caching mechanisms.

Static analysis via SonarQube confirms the application is secure, passing the quality gate with zero vulnerabilities. However, the report highlights areas for refactoring to reduce technical debt. With

12.9% code duplication and 35 code smells, the primary focus for future development should be on improving maintainability through code simplification and the extraction of shared logic.

In summary, the REST API functions reliably within tested parameters. To prepare the application for production-grade environments, future work should focus on two key areas: refactoring the codebase to eliminate code duplication and investigating different connection strategies to overcome the TCP connection limits identified during high volume testing.